

Homework 1

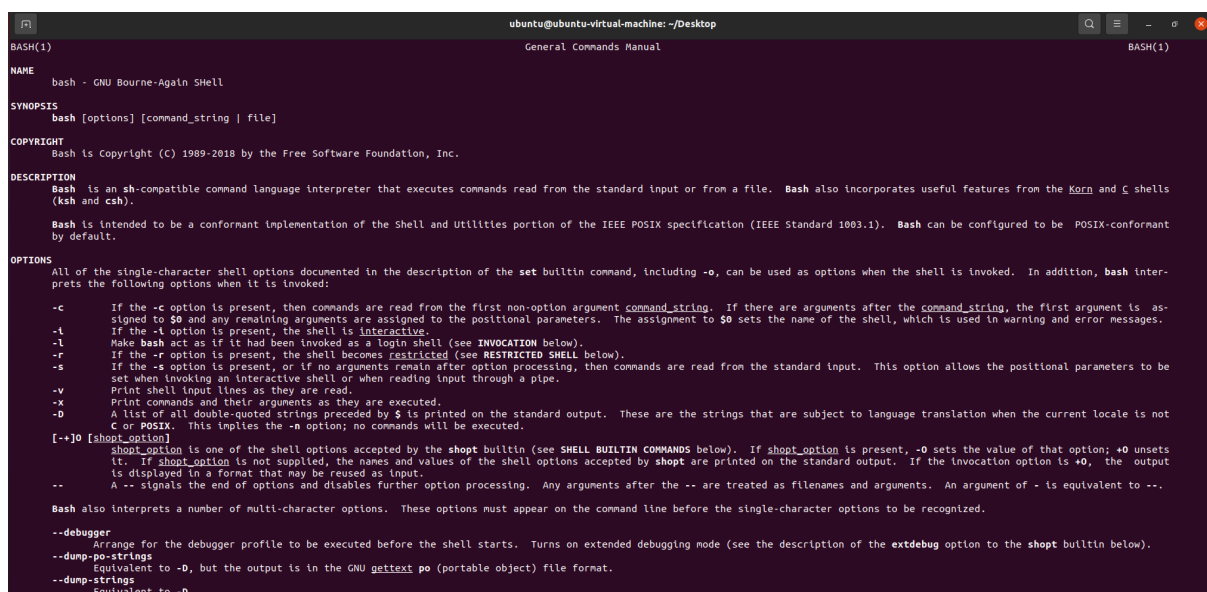
B113040045 許育菴

Q1: Who is the author of bash shell?

Ans: Its author is Brian Fox, but I can't use `man bash` to find the author name.

I find the answer from wikipedia -

[https://en.wikipedia.org/wiki/Bash_\(Unix_shell\)](https://en.wikipedia.org/wiki/Bash_(Unix_shell)) .

A screenshot of a terminal window showing the output of the 'man bash' command. The window title is 'ubuntu@ubuntu-virtual-machine: ~/Desktop'. The terminal displays the 'General Commands Manual' for 'BASH(1)'. The content includes sections for NAME, SYNOPSIS, COPYRIGHT, DESCRIPTION, and OPTIONS. The DESCRIPTION section states that Bash is a sh-compatible command language interpreter. The OPTIONS section lists various flags like -c, -l, -r, -s, -v, -x, -D, and shopt options, along with multi-character options like --debugger, --dump-po-strings, and --dump-strings.

Screen shot of `man bash` .

Q2: What is the retrun type of the `read()` system call?

Ans: `ssize_t` , it is a integer.

```
READ(2)                                                                 Linux Programmer's Manual                                                                 READ(2)

NAME
    read - read from a file descriptor

SYNOPSIS
    #include <unistd.h>

    ssize_t read(int fd, void *buf, size_t count);

DESCRIPTION
    read() attempts to read up to count bytes from file descriptor fd into the buffer starting at buf.

    On files that support seeking, the read operation commences at the file offset, and the file offset is incremented by the number of bytes read. If the file offset is at or past the end of file, no bytes are read, and read() returns zero.

    If count is zero, read() may detect the errors described below. In the absence of any errors, or if read() does not check for errors, a read() with a count of 0 returns zero and has no other effects.

    According to POSIX.1, if count is greater than SSIZE_MAX, the result is implementation-defined; see NOTES for the upper limit on Linux.

RETURN VALUE
    On success, the number of bytes read is returned (zero indicates end of file), and the file position is advanced by this number. It is not an error if this number is smaller than the number of bytes requested; this may happen for example because fewer bytes are actually available right now (maybe because we were close to end-of-file, or because we are reading from a pipe, or from a terminal), or because read() was interrupted by a signal. See also NOTES.

    On error, -1 is returned, and errno is set appropriately. In this case, it is left unspecified whether the file position (if any) changes.
```

Screen shot of `man read`.

Q3: Using the man pages, find the names of all of the header files that you would need to include to use the following functions in a program. There might be more than one needed for some of these.

Ans: Use `man` + each function name to get the include library information.

- a. `_exit()` : `#include <unistd.h>`

```
_EXIT(2)

NAME
    _exit, _Exit - terminate the calling process

SYNOPSIS
    #include <unistd.h>

    void _exit(int status);

    #include <stdlib.h>

    void _Exit(int status);

Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    _Exit():
        _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L
```

- b. `setuid()` : `#include<sys/types.h>` `#include <unistd.h>`

```
SETUID(2) Linux Program

NAME
    setuid - set user identity

SYNOPSIS
    #include <sys/types.h>
    #include <unistd.h>

    int setuid(uid_t uid);

DESCRIPTION
    setuid() sets the effective user ID of the calling process. If the calling process is pr
    real UID and saved set-user-ID are also set.

    Under Linux, setuid() is implemented like the POSIX version with the _POSIX_SAVED_IDS featur
    some un-privileged work, and then reengage the original effective user ID in a secure manne

    If the user is root or the program is set-user-ID-root, special care must be taken: setuid
    ID's are set to uid. After this has occurred, it is impossible for the program to regain r

    Thus, a set-user-ID-root program wishing to temporarily drop root privileges, assume the id
    You can accomplish this with seteuid(2).
```

- C. `fstat()` : `#include<sys/types.h>` `#include <sys/stat.h>` `#include <unistd.h>`

```
STAT(2)

NAME
    stat, fstat, lstat, fstatat - get file status

SYNOPSIS
    #include <sys/types.h>
    #include <sys/stat.h>
    #include <unistd.h>

    int stat(const char *pathname, struct stat *statbuf);
    int fstat(int fd, struct stat *statbuf);
    int lstat(const char *pathname, struct stat *statbuf);

    #include <fcntl.h>          /* Definition of AT_* constants */
    #include <sys/stat.h>

    int fstatat(int dirfd, const char *pathname, struct stat *statbuf,
                int flags);
```

Q4: If your current working directory is `/usr/share/gcc/python`, what is the shortest relative pathname of the file `/usr/lib32/libc.so.6`?

Ans: `../../lib32/libc.so.6`

Q5: What command can be used to print the creation date of a file?

Ans: `stat <filename>` or `stat -c %w <filename>`, the creation date of file will be show behind "Birth" text.

However, some filesystems don't support recording the creation date of files, if the filesystem doesn't support this function, it will show "-" behind "Birth" text.

```
ubuntu@ubuntu-virtual-machine:~/Desktop/HW2$ touch test.txt
ubuntu@ubuntu-virtual-machine:~/Desktop/HW2$ stat test.txt
  File: test.txt
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 805h/2053d Inode: 795580       Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2026-03-17 22:03:44.070743074 +0800
Modify: 2026-03-17 22:03:44.070743074 +0800
Change: 2026-03-17 22:03:44.070743074 +0800
 Birth: -
ubuntu@ubuntu-virtual-machine:~/Desktop/HW2$ stat -c %w test.txt
-
ubuntu@ubuntu-virtual-machine:~/Desktop/HW2$
```

Use `stat` command to show the creation date of a file.